Lab 06: Network Introduction

Operating Systems and Distributed Systems

This lab aims to introduce tools that provide valuable information when working with networks. It also enforces the students' understanding of the layers in the OSI and TCP/IP models.

The lab is structured so the student is assisted through the tasks.

Remember to read the exercise and the accompanying text thoroughly before asking for help.

Contents

Exercise 01: Facebook Developer

In this exercise, the student will examine the Facebook API to understand how a (REST) API works and the capabilities that an API like this offers.

> **Hint 1**
>
> The full documentation can be found at https://developers.facebook.com/docs/graph-api/overview

Facebook Developer Graph API Explorer

The Facebook Developer Graph API Explorer [https://developers.facebook.com/tools/explorer/] is a tool for testing, creating and debugging API calls.

To use the tool, it is required to create an app to tell Facebook that you would now like to use Facebook's API. This makes you responsible for every call made through the app.

With an app created, API Explorer can be used. A User token is enabled by default, making it possible for the application to identify the user only by name and ID. If access to other information is needed, permission for the desired information must be included.

The access token identifies and authorises the application's user, allowing us to use the API in an appropriate context.

The Facebook API is accessible through https://graph.facebook.com/. Remember that REST APIs use endpoints, so the access point mentioned above needs to be specified.

cURL

The Linux Terminal tool curl is used to transfer data using several protocols. This tool is helpful as we use the HTTPS protocol for the Facebook API.

We use cURL to communicate with an HTTPS server with a GET method, requesting some data about the app's user.

The tool, like many other Linux tools, is used in the following fashion: `$ curl <OPTIONS> <URL-ENDPOINT>`. The only option we are interested in for now is `-H` (or `--header`), which is used to include a header to the request.

When using cURL to access the Facebook API, you must include your token in the request's header. To authorise yourself with the token, use the following header: `"Authorization: Bearer <TOKEN>"`.

Your task

1. Create a Facebook app through Facebook for Developers [https://developers.facebook.com]

(a) Select ”Log In” from the menu and log in using your Facebook credentials

(b) Select ”My Apps / Apps” from the menu and select ”Create App”

(c) Select ”I don’t want to connect a business portfolio yet.”

(d) Select ”Other”

(e) Select ”Consumer”

(f) Give the app a name and select ”Create App”

2. Open the Graph API Explorer through ”Tools” in the menu
3. Select ”Get Token” and ”Get User Access Token”
4. Include permission for `user_birthday` through the API Explorer
5. Click ”Generate Access Token” and grab your token for authorisation when using cURL
6. Construct a URL requesting your birthday from the Facebook API through the appropriate endpoint
7. Use cURL to create a `GET` request with a custom header, making Facebook authorise your request and respond with your birth date

The exercise will return the following message:

```
{
  "birthday": "01/01/2000",
  "id": "00000000000000000"
}
```

Exercise 02: Wireshark

In this exercise, you will use Wireshark to inspect network traffic.

> **Information 1**
>
> You can filter based on the TCP flags (for example: `tcp.flags==0x02`) and specific protocols like `arp` and `dns`.

Your task

1. Download Wireshark from https://www.wireshark.org/download.html
2. Open Wireshark from the terminal with `$ sudo wireshark`
3. Start capturing on the interface connected to the internet (most likely a wifi interface)
4. Visit a website, for example, https://sdu.dk
5. Stop capturing in Wireshark
6. Use filtering in Wireshark to see only SYN, SYN/ACK and ACK messages
7. Can you explain what is happening?
8. Can you find any ARP messages? Why/why not?
9. Can you find any DNS messages? Why/why not?

tcp.flags.syn == 1 and tcp.flags.ack == 0 (SYN only)
tcp.flags.syn == 1 and tcp.flags.ack == 1 (SYN/ACK only)
tcp.flags.syn == 0 and tcp.flags.ack == 1 (ACK only)

7. What is happening?
You are seeing the TCP three-way handshake:
SYN – your computer requests a connection.
SYN/ACK – the server agrees.
ACK – your computer confirms.
After that, data transfer (HTTPS) begins.

8. ARP messages
You might not see ARP packets because your computer already knows the MAC addresses on the local network (they're cached). ARP is only used inside the LAN to find the hardware address of an IP.

9. DNS messages
You may see or not see DNS packets. If they're missing, it's likely because the DNS result is cached or your browser uses encrypted DNS (DoH/DoT), so normal DNS traffic isn't visible.

Exercise 03: Hello, Flask!

In this exercise, the student will create a server using python and a package named Flask. Using Flask, creating a custom API and setting up specific routes is possible. An example of a simple Flask app can be seen below.

```
from flask import Flask

# This sets up the application using the Flask object from the package flask.
app = Flask(__name__)

# Using decorators the route is associated with the following method.
@app.route('/')
def hello():
  return 'Greetings sire, the Docker universe awaits you!'

# This statement evaluates to true if this is the main python file. It starts up
    the Flask app on localhost with the default port 5000.
if __name__ == '__main__':
  app.run(host='0.0.0.0')
```

In this task, `python3` will be used. Installing packages can quickly be done using the software `pip`, and when working with `python3` on Linux, it is called `pip3`. To check whether `pip` is installed, use the command `$ pip3 --version`. If not, run `$ apt-get install python3-pip`. To install the package `Flask`, run `$ pip3 install Flask`.

Information 2

More information about the Flask framework can be found here: https://flask.palletsprojects.com/en/2.3.x/quickstart/

Your task

1. Create a server in Python using flask
2. Enable the route /welcome and give it the response: *"Welcome to this excellent page!"*
3. Run the script and access it at http://localhost:5000/welcome

The exercise will return the following message:

```
1 Welcome to this excellent page!
```

Exercise 04: Hello, <NAME>!

In this exercise, the student will use the previously created server and run it in a container.

Information 3

More information on using the Python image can be found on Docker Hub: https://hub.docker.com/_/python

Information 4

Installing Flask in a container can be done similarly to installing it on Linux

Hint 2

The server is exposing port 5000 in the container.

Your task

1. Modify the server from the previous exercise
   (a) Extend the route `/welcome` to take a name as a parameter
   (b) Make the server respond with: *"Welcome to this excellent page, <NAME>!"* where `<NAME>` is replaced with the name argument
2. Create a Dockerfile that
   (a) Uses the Python image as base
   (b) Installs the package Flask
   (c) Copies the server file
   (d) Runs the Python server
3. Build an image from the Dockerfile. Remember to give the build a tag
4. Run the built image and forward the container's port to port 7000
5. Access the server running in the container at http://localhost:7000/welcome/James

The exercise will return the following message:

```
1 Welcome to this excellent page, James!
```

Exercise 05: Public Geocoding API

In this exercise, the student will contact a Geocoding API that, given a postcode, returns information about the city. https://hippopotam.us provide the API. Other public APIs can be found here: https://github.com/public-apis/public-apis/blob/master/README.md.

The Danish information can be accessed through `https://api.zippopotam.us/DK/<POSTCODE>`.

The API returns a JSON object with the related information.

Your task

1. Use cURL and the endpoint mentioned above to get information about Odense.
2. Examine the returned JSON object and take notice of the structure.

Exercise 06: Public Weather API

In this exercise, the student will contact a weather API to get the temperature of a given latitude and longitude. A public API for weather is Open-Meteo, which has several routes and options.

Using their forecast endpoint, `https://api.open-meteo.com/v1/forecast?timezone=auto&daily=temperature_2m_max,temperature_2m_min`, it is possible to get a forecast of the coming days' minimum and maximum temperatures.

To get the information for a specific location, the endpoint have to be extended with additional information like latitude and longitude: `https://api.open-meteo.com/v1/forecast?timezone=auto&daily=temperature_2m_max,temperature_2m_min&latitude=<LAT>&longitude=<LONG>`.

Hint 3

For precise information, use the longitude and latitude from the previous exercise.

Your task

1. Use cURL and the endpoint mentioned above to get information about Odense.
2. Examine the returned JSON object and take notice of the structure.

Exercise 07: Public APIs – Combined

In this exercise, the student will use the server previously created and the two APIs to create a custom API that returns the temperature of a requested postcode.

Python and JSON

Python can request data and interpret it as JSON using the following method:

```
# imports the necessary libraries. If not available, the package can be installed
    using pip
import requests

def get_data_from_url_request(url_endpoint):
  # Make a GET request and save the response
  response = requests.get(url_endpoint)
  # Interpret the response as json
  data = response.json()
  return data
```

Hint 4

The server is exposing port 5000 in the container.

Your task

1. Modify the server from the previous exercise
   (a) Create a route `/weather` which takes a postcode as argument
   (b) The route should respond with: *The temperature in* `<CITY>` *will be between* `<TEMP_MIN>` *and* `<TEMP_MAX>` *tomorrow*, where `<CITY>`, `<TEMP_MIN>` and `TEMP_MAX` are values returned by the two APIs.
2. Use the postcode to receive latitude, longitude and city name
3. Use the retrieved latitude and longitude to request the temperature
4. Use the Dockerfile from Exercise 3 to build an image. Remember to give the build a tag.
5. Run the built image and forward the container's port to port 7000.
6. Get the current temperature in Odense by accessing the server running in the container at http://localhost:5000/weather/5000

The exercise will return the following message:

```
1 The temperature in Odense C will be between 11.3 and 14.6 tomorrow
```